3_4_integrated_pipeline

March 10, 2026

[1]:
```
import os
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
from sklearn.metrics import mean_squared_error, mean_absolute_error
from scipy import stats
from scipy.stats import ks_2samp, anderson_ksamp
from torch.utils.data import DataLoader, TensorDataset
import warnings
warnings.filterwarnings('ignore')

# Your data directory setup
os.getcwd()
```

[1]:
```
'/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-data'
```

[2]:
```
os.chdir(path = "data/processed_files")
```

0.0.1 BaseGAN Class

[3]:
```
class BaseGAN:
    def __init__(self, config):
        self.config = config

    def train(self, train_data):
        raise NotImplementedError

    def validate(self, val_data):
        raise NotImplementedError

    def generate(self, n_samples):
        raise NotImplementedError
```

```
    def name(self):
        return self.__class__.__name__
```

0.0.2 TimeGAN

[4]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import pandas as pd
from torch.utils.data import DataLoader, TensorDataset

# -------------------------------
# Embedder Network (RNN-based)
# -------------------------------
class Embedder(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_layers=3):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers

        self.rnn = nn.GRU(
            input_dim, hidden_dim, num_layers,
            batch_first=True
        )
        self.fc = nn.Linear(hidden_dim, hidden_dim)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        # x: (batch, seq_len, input_dim)
        h, _ = self.rnn(x)
        h = self.sigmoid(self.fc(h))
        return h

# -------------------------------
# Recovery Network (RNN-based)
# -------------------------------
class Recovery(nn.Module):
    def __init__(self, hidden_dim, output_dim, num_layers=3):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers

        self.rnn = nn.GRU(
            hidden_dim, hidden_dim, num_layers,
            batch_first=True
        )
```

```python
        self.fc = nn.Linear(hidden_dim, output_dim)
        self.sigmoid = nn.Sigmoid()

    def forward(self, h):
        # h: (batch, seq_len, hidden_dim)
        x, _ = self.rnn(h)
        x = self.sigmoid(self.fc(x))
        return x

# -------------------------------
# Generator Network (RNN-based)
# -------------------------------
class Generator(nn.Module):
    def __init__(self, noise_dim, hidden_dim, num_layers=3):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers

        self.rnn = nn.GRU(
            noise_dim, hidden_dim, num_layers,
            batch_first=True
        )
        self.fc = nn.Linear(hidden_dim, hidden_dim)
        self.sigmoid = nn.Sigmoid()

    def forward(self, z):
        # z: (batch, seq_len, noise_dim)
        h, _ = self.rnn(z)
        h = self.sigmoid(self.fc(h))
        return h

# -------------------------------
# Supervisor Network (for step-ahead prediction)
# -------------------------------
class Supervisor(nn.Module):
    def __init__(self, hidden_dim, num_layers=2):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers

        self.rnn = nn.GRU(
            hidden_dim, hidden_dim, num_layers,
            batch_first=True
        )
        self.fc = nn.Linear(hidden_dim, hidden_dim)
        self.sigmoid = nn.Sigmoid()
```

```python
    def forward(self, h):
        # h: (batch, seq_len, hidden_dim)
        h_out, _ = self.rnn(h)
        h_out = self.sigmoid(self.fc(h_out))
        return h_out

# -------------------------------
# Discriminator Network (RNN-based)
# -------------------------------
class Discriminator(nn.Module):
    def __init__(self, hidden_dim, num_layers=3):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers

        self.rnn = nn.GRU(
            hidden_dim, hidden_dim, num_layers,
            batch_first=True
        )
        self.fc = nn.Linear(hidden_dim, 1)

    def forward(self, h):
        # h: (batch, seq_len, hidden_dim)
        h_out, _ = self.rnn(h)
        y = self.fc(h_out)  # (batch, seq_len, 1)
        return y

# -------------------------------
# TimeGAN Implementation
# -------------------------------
class TimeGAN(BaseGAN):
    def __init__(self, config=None):
        super().__init__(config or {})
        cfg = self.config

        self.device = torch.device("cuda" if torch.cuda.is_available() else␣
 ↪"cpu")

        # Hyperparameters
        self.hidden_dim = cfg.get("hidden_dim", 24)
        self.noise_dim = cfg.get("noise_dim", 24)
        self.num_layers = cfg.get("num_layers", 3)
        self.lr = cfg.get("lr", 1e-3)
        self.batch_size = cfg.get("batch_size", 128)
        self.epochs = cfg.get("epochs", 50)

        # Training iterations for each phase
```

```
        self.iterations = cfg.get("iterations", 10000)

        # Data-dependent
        self.seq_len = None
        self.n_features = None
        self.data_min = None
        self.data_max = None
        self.data_mean = None
        self.data_std = None

        # Networks
        self.embedder = None
        self.recovery = None
        self.generator = None
        self.supervisor = None
        self.discriminator = None

        # Optimizers
        self.opt_autoencoder = None
        self.opt_supervisor = None
        self.opt_generator = None
        self.opt_discriminator = None

    def _normalize_data(self, data, fit=True):
        if fit:
            self.data_min = np.min(data, axis=(0, 1), keepdims=True)
            self.data_max = np.max(data, axis=(0, 1), keepdims=True)
            self.data_mean = np.mean(data, axis=(0, 1), keepdims=True)
            self.data_std = np.std(data, axis=(0, 1), keepdims=True) + 1e-8

            print(f"[TimeGAN] Data range: [{data.min():.4f}, {data.max():.4f}]")
            print(f"[TimeGAN] Mean: {self.data_mean.squeeze():.6f}, Std: {self.
↪data_std.squeeze():.6f}")

        # Min-max normalization to [0, 1] (TimeGAN uses sigmoid activations)
        data_range = self.data_max - self.data_min + 1e-8
        normalized = (data - self.data_min) / data_range
        normalized = np.clip(normalized, 0.0, 1.0)

        if fit:
            print(f"[TimeGAN] Normalized range: [{normalized.min():.4f},␣
↪{normalized.max():.4f}]")

        return normalized

    def _denormalize_data(self, data):
        data_range = self.data_max - self.data_min
```

```
        return data * data_range + self.data_min

    def _make_windows(self, series, seq_len):
        T = series.shape[0]
        if T < seq_len:
            raise ValueError(f"Time series too short ({T}) for␣
↪seq_len={seq_len}")

        windows = []
        for i in range(T - seq_len + 1):
            windows.append(series[i:i+seq_len])

        return np.array(windows)

    def _build_models(self, seq_len, n_features):
        self.seq_len = seq_len
        self.n_features = n_features

        # Build networks
        self.embedder = Embedder(n_features, self.hidden_dim, self.num_layers).
↪to(self.device)
        self.recovery = Recovery(self.hidden_dim, n_features, self.num_layers).
↪to(self.device)
        self.generator = Generator(self.noise_dim, self.hidden_dim, self.
↪num_layers).to(self.device)
        self.supervisor = Supervisor(self.hidden_dim, 2).to(self.device)
        self.discriminator = Discriminator(self.hidden_dim, self.num_layers).
↪to(self.device)

        # Optimizers
        self.opt_autoencoder = optim.Adam(
            list(self.embedder.parameters()) + list(self.recovery.parameters()),
            lr=self.lr
        )
        self.opt_supervisor = optim.Adam(
            list(self.supervisor.parameters()) + list(self.generator.
↪parameters()),
            lr=self.lr
        )
        self.opt_generator = optim.Adam(
            list(self.generator.parameters()) + list(self.supervisor.
↪parameters()),
            lr=self.lr
        )
        self.opt_discriminator = optim.Adam(
            self.discriminator.parameters(),
```

```
            lr=self.lr
        )

        print(f"[TimeGAN] Models built:")
        print(f"  Embedder: {sum(p.numel() for p in self.embedder.parameters()):
↪,}")
        print(f"  Recovery: {sum(p.numel() for p in self.recovery.parameters()):
↪,}")
        print(f"  Generator: {sum(p.numel() for p in self.generator.
↪parameters()):,}")
        print(f"  Supervisor: {sum(p.numel() for p in self.supervisor.
↪parameters()):,}")
        print(f"  Discriminator: {sum(p.numel() for p in self.discriminator.
↪parameters()):,}")

    def train(self, train_data):
        data = np.asarray(train_data, dtype=np.float32)

        if data.ndim == 1:
            data = data[:, None]

        # Handle NaN
        if np.isnan(data).any():
            nan_count = np.isnan(data).sum()
            print(f"[TimeGAN] WARNING: Found {nan_count} NaN values, handling...
↪")
            data_df = pd.DataFrame(data)
            data_df = data_df.fillna(method='ffill').fillna(method='bfill').
↪fillna(0.0)
            data = data_df.values
            print(f"[TimeGAN] NaN handling complete")

        T, n_features = data.shape

        # Choose seq_len
        max_seq_len = T
        if max_seq_len >= 256:
            seq_len = 256
        elif max_seq_len >= 128:
            seq_len = 128
        elif max_seq_len >= 64:
            seq_len = 64
        elif max_seq_len >= 32:
            seq_len = 32
        elif max_seq_len >= 24:
            seq_len = 24
```

```
        else:
            seq_len = max(16, max_seq_len)

        print(f"[TimeGAN] Creating windows: T={T}, seq_len={seq_len},␣
↪n_features={n_features}")

        data = self._make_windows(data, seq_len)
        data = self._normalize_data(data, fit=True)

        n_windows = data.shape[0]
        print(f"[TimeGAN] Created {n_windows} windows")

        if self.embedder is None:
            self._build_models(seq_len, n_features)

        # Convert to tensors
        data_tensor = torch.tensor(data, dtype=torch.float32).to(self.device)

        # ====================================
        # Phase 1: Train Embedder & Recovery
        # ====================================
        print("[TimeGAN] Phase 1: Training Autoencoder...")
        for epoch in range(self.epochs // 2):
            self.embedder.train()
            self.recovery.train()

            # Sample random batch
            idx = np.random.permutation(n_windows)[:self.batch_size]
            X = data_tensor[idx]

            # Forward
            H = self.embedder(X)
            X_tilde = self.recovery(H)

            # Reconstruction loss
            loss_reconstruction = nn.MSELoss()(X, X_tilde)

            # Backward
            self.opt_autoencoder.zero_grad()
            loss_reconstruction.backward()
            torch.nn.utils.clip_grad_norm_(
                list(self.embedder.parameters()) + list(self.recovery.
↪parameters()),
                max_norm=1.0
            )
            self.opt_autoencoder.step()
```

```
            if (epoch + 1) % 10 == 0 or epoch == 0:
                print(f"  Epoch {epoch+1}/{self.epochs//2} | Reconstruction␣
↪Loss: {loss_reconstruction.item():.4f}")

        # ====================================
        # Phase 2: Train Supervisor
        # ====================================
        print("[TimeGAN] Phase 2: Training Supervisor...")
        for epoch in range(self.epochs // 2):
            self.supervisor.train()
            self.generator.train()

            idx = np.random.permutation(n_windows)[:self.batch_size]
            X = data_tensor[idx]

            # Generate embeddings
            with torch.no_grad():
                H = self.embedder(X)

            # Supervised loss: predict next step
            H_supervise = self.supervisor(H)
            loss_supervisor = nn.MSELoss()(H[:, 1:, :], H_supervise[:, :-1, :])

            # Backward
            self.opt_supervisor.zero_grad()
            loss_supervisor.backward()
            torch.nn.utils.clip_grad_norm_(
                list(self.supervisor.parameters()) + list(self.generator.
↪parameters()),
                max_norm=1.0
            )
            self.opt_supervisor.step()

            if (epoch + 1) % 10 == 0 or epoch == 0:
                print(f"  Epoch {epoch+1}/{self.epochs//2} | Supervisor Loss:␣
↪{loss_supervisor.item():.4f}")

        # ====================================
        # Phase 3: Joint Training (GAN)
        # ====================================
        print("[TimeGAN] Phase 3: Joint Adversarial Training...")
        for epoch in range(self.epochs):
            g_losses = []
            d_losses = []

            for _ in range(min(6, n_windows // self.batch_size)):
                idx = np.random.permutation(n_windows)[:self.batch_size]
```

```
                X = data_tensor[idx]

                # ===== Train Discriminator =====
                self.discriminator.train()

                # Real data
                with torch.no_grad():
                    H_real = self.embedder(X)

                # Fake data
                Z = torch.randn(self.batch_size, self.seq_len, self.noise_dim).
↪to(self.device)
                with torch.no_grad():
                    E_hat = self.generator(Z)
                    H_hat = self.supervisor(E_hat)

                # Discriminator loss
                y_real = self.discriminator(H_real)
                y_fake = self.discriminator(H_hat)

                d_loss_real = nn.BCEWithLogitsLoss()(y_real, torch.
↪ones_like(y_real))
                d_loss_fake = nn.BCEWithLogitsLoss()(y_fake, torch.
↪zeros_like(y_fake))
                d_loss = d_loss_real + d_loss_fake

                self.opt_discriminator.zero_grad()
                d_loss.backward()
                torch.nn.utils.clip_grad_norm_(self.discriminator.parameters(),␣
↪max_norm=1.0)
                self.opt_discriminator.step()

                d_losses.append(d_loss.item())

                # ===== Train Generator =====
                self.generator.train()
                self.supervisor.train()

                # Generate fake data
                Z = torch.randn(self.batch_size, self.seq_len, self.noise_dim).
↪to(self.device)
                E_hat = self.generator(Z)
                H_hat = self.supervisor(E_hat)

                # Reconstruct
                with torch.no_grad():
                    H_real = self.embedder(X)
```

```
                H_supervise = self.supervisor(H_real)

                # Generator losses
                y_fake_g = self.discriminator(H_hat)
                g_loss_u = nn.BCEWithLogitsLoss()(y_fake_g, torch.
↪ones_like(y_fake_g))
                g_loss_s = nn.MSELoss()(H_real[:, 1:, :], H_supervise[:, :-1, :
↪])

                # Moment matching loss
                g_loss_v1 = torch.mean(torch.abs(torch.mean(H_real, dim=0) -␣
↪torch.mean(H_hat, dim=0)))
                g_loss_v2 = torch.mean(torch.abs(torch.std(H_real, dim=0) -␣
↪torch.std(H_hat, dim=0)))
                g_loss_v = g_loss_v1 + g_loss_v2

                # Total generator loss
                g_loss = g_loss_u + 100 * torch.sqrt(g_loss_s) + 100 * g_loss_v

                self.opt_generator.zero_grad()
                g_loss.backward()
                torch.nn.utils.clip_grad_norm_(
                    list(self.generator.parameters()) + list(self.supervisor.
↪parameters()),
                    max_norm=1.0
                )
                self.opt_generator.step()

                g_losses.append(g_loss.item())

            if len(d_losses) > 0 and len(g_losses) > 0:
                avg_d = np.mean(d_losses)
                avg_g = np.mean(g_losses)

                if (epoch + 1) % 5 == 0 or epoch == 0:
                    print(f"[TimeGAN] Epoch {epoch+1}/{self.epochs} | D={avg_d:.
↪4f} | G={avg_g:.4f}")

    def validate(self, val_data):
        data = np.asarray(val_data, dtype=np.float32)

        if data.ndim == 1:
            data = data[:, None]

        if np.isnan(data).any():
            data_df = pd.DataFrame(data)
```

```
            data_df = data_df.fillna(method='ffill').fillna(method='bfill').
↪fillna(0.0)
            data = data_df.values

        if self.embedder is None:
            raise RuntimeError("Model not trained yet")

        T = data.shape[0]

        if T < self.seq_len:
            temp_seq_len = max(16, T)
            if temp_seq_len < 16:
                return 0.0

            data = self._make_windows(data, temp_seq_len)
            data = self._normalize_data(data, fit=False)

            n_windows, _, n_features = data.shape
            padded_data = np.zeros((n_windows, self.seq_len, n_features),␣
↪dtype=np.float32)
            padded_data[:, :temp_seq_len, :] = data
            data = padded_data
        else:
            data = self._make_windows(data, self.seq_len)
            data = self._normalize_data(data, fit=False)

        real = torch.tensor(data, dtype=torch.float32).to(self.device)

        self.embedder.eval()
        self.discriminator.eval()

        with torch.no_grad():
            batch_size = min(self.batch_size, real.size(0))
            H_real = self.embedder(real[:batch_size])
            y_real = self.discriminator(H_real)
            score = torch.sigmoid(y_real).mean().item()

        self.embedder.train()
        self.discriminator.train()

        return score

    def generate(self, n_samples):
        if self.generator is None:
            raise RuntimeError("Model not trained yet")

        self.generator.eval()
```

```
        self.supervisor.eval()
        self.recovery.eval()

        n_windows = max(1, (n_samples + self.seq_len - 1) // self.seq_len)

        out = []
        with torch.no_grad():
            for i in range(0, n_windows, self.batch_size):
                b = min(self.batch_size, n_windows - i)
                Z = torch.randn(b, self.seq_len, self.noise_dim).to(self.device)
                E_hat = self.generator(Z)
                H_hat = self.supervisor(E_hat)
                X_hat = self.recovery(H_hat)
                out.append(X_hat.cpu().numpy())

        self.generator.train()
        self.supervisor.train()
        self.recovery.train()

        windows = np.concatenate(out, axis=0)
        windows = self._denormalize_data(windows)

        reconstructed = windows.reshape(-1, self.n_features)
        reconstructed = reconstructed[:n_samples]

        if reconstructed.shape[0] < n_samples:
            padding = np.repeat(reconstructed[-1:], n_samples - reconstructed.
↪shape[0], axis=0)
            reconstructed = np.vstack([reconstructed, padding])

        return reconstructed
```

0.0.3 QuantGAN

[5]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import pandas as pd
from torch.utils.data import DataLoader, TensorDataset

# -------------------------------
# Causal Convolution with proper length preservation
# -------------------------------
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation=1):
        super().__init__()
```

```
        self.padding = (kernel_size - 1) * dilation
        self.conv = nn.Conv1d(in_channels, out_channels, kernel_size,
                              padding=self.padding, dilation=dilation)

    def forward(self, x):
        # Apply conv with left padding
        out = self.conv(x)
        # Remove right padding to maintain causality and original length
        if self.padding > 0:
            out = out[:, :, :-self.padding]
        return out

# -------------------------------
# Temporal Block with Causal Convolutions
# -------------------------------
class TemporalBlock(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation,␣
 ↪dropout=0.2):
        super().__init__()

        self.conv1 = CausalConv1d(in_channels, out_channels, kernel_size,␣
 ↪dilation)
        self.bn1 = nn.BatchNorm1d(out_channels)
        self.relu1 = nn.ReLU()
        self.dropout1 = nn.Dropout(dropout)

        self.conv2 = CausalConv1d(out_channels, out_channels, kernel_size,␣
 ↪dilation)
        self.bn2 = nn.BatchNorm1d(out_channels)
        self.relu2 = nn.ReLU()
        self.dropout2 = nn.Dropout(dropout)

        self.net = nn.Sequential(
            self.conv1, self.bn1, self.relu1, self.dropout1,
            self.conv2, self.bn2, self.relu2, self.dropout2
        )

        self.downsample = nn.Conv1d(in_channels, out_channels, 1) if␣
 ↪in_channels != out_channels else None
        self.relu = nn.ReLU()

    def forward(self, x):
        out = self.net(x)
        res = x if self.downsample is None else self.downsample(x)
        return self.relu(out + res)

# -------------------------------
```

```python
# Temporal Convolutional Network
# -------------------------------
class TemporalConvNet(nn.Module):
    def __init__(self, num_inputs, num_channels, kernel_size=3, dropout=0.2):
        super().__init__()
        layers = []
        num_levels = len(num_channels)

        for i in range(num_levels):
            dilation_size = 2 ** i
            in_channels = num_inputs if i == 0 else num_channels[i-1]
            out_channels = num_channels[i]

            layers.append(TemporalBlock(
                in_channels, out_channels, kernel_size,
                dilation=dilation_size, dropout=dropout
            ))

        self.network = nn.Sequential(*layers)

    def forward(self, x):
        return self.network(x)

# -------------------------------
# QuantGAN Generator (TCN-based)
# -------------------------------
class QuantGAN_Generator(nn.Module):
    def __init__(self, noise_dim, seq_len, n_features, hidden_channels=[64, 64,␣
 ↪32]):
        super().__init__()
        self.seq_len = seq_len
        self.n_features = n_features

        # Project noise to initial sequence
        self.fc = nn.Linear(noise_dim, hidden_channels[0] * (seq_len // 4))

        # Upsample to target length
        self.upsample1 = nn.ConvTranspose1d(hidden_channels[0],␣
 ↪hidden_channels[0], 4, stride=2, padding=1)
        self.bn1 = nn.BatchNorm1d(hidden_channels[0])
        self.relu1 = nn.ReLU()

        self.upsample2 = nn.ConvTranspose1d(hidden_channels[0],␣
 ↪hidden_channels[1], 4, stride=2, padding=1)
        self.bn2 = nn.BatchNorm1d(hidden_channels[1])
        self.relu2 = nn.ReLU()
```

```python
        # TCN for temporal dependencies
        self.tcn = TemporalConvNet(
            num_inputs=hidden_channels[1],
            num_channels=hidden_channels[1:],
            kernel_size=3,
            dropout=0.2
        )

        # Final projection
        self.output = nn.Sequential(
            nn.Conv1d(hidden_channels[-1], n_features, 1),
            nn.Tanh()
        )

    def forward(self, z):
        batch_size = z.size(0)

        # Project and reshape
        x = self.fc(z)
        x = x.view(batch_size, -1, self.seq_len // 4)

        # Upsample
        x = self.relu1(self.bn1(self.upsample1(x)))
        x = self.relu2(self.bn2(self.upsample2(x)))

        # Ensure correct length
        if x.size(2) != self.seq_len:
            x = x[:, :, :self.seq_len]

        # Apply TCN (maintains length due to causal padding)
        x = self.tcn(x)

        # Final adjustment to ensure exact length
        if x.size(2) != self.seq_len:
            x = x[:, :, :self.seq_len]

        # Output
        x = self.output(x)

        return x.permute(0, 2, 1)

# -------------------------------
# QuantGAN Discriminator (TCN-based)
# -------------------------------
class QuantGAN_Discriminator(nn.Module):
    def __init__(self, seq_len, n_features, hidden_channels=[32, 64, 128]):
        super().__init__()
```

```
        self.tcn = TemporalConvNet(
            num_inputs=n_features,
            num_channels=hidden_channels,
            kernel_size=3,
            dropout=0.2
        )

        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool1d(1),
            nn.Flatten(),
            nn.Linear(hidden_channels[-1], 64),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),
            nn.Linear(64, 1)
        )

    def forward(self, x):
        x = x.permute(0, 2, 1)
        x = self.tcn(x)
        x = self.classifier(x)
        return x

# -------------------------------
# QuantGAN (WGAN-GP with TCN)
# -------------------------------
class QuantGAN(BaseGAN):
    def __init__(self, config=None):
        super().__init__(config or {})
        cfg = self.config

        self.device = torch.device("cuda" if torch.cuda.is_available() else␣
 ↪"cpu")

        self.noise_dim = cfg.get("noise_dim", 100)
        self.lr_g = cfg.get("lr_g", 1e-4)
        self.lr_d = cfg.get("lr_d", 1e-4)
        self.batch_size = cfg.get("batch_size", 64)
        self.epochs = cfg.get("epochs", 50)
        self.lambda_gp = cfg.get("lambda_gp", 10.0)
        self.n_critic = cfg.get("n_critic", 3)
        self.hidden_channels_g = cfg.get("hidden_channels_g", [64, 64, 32])
        self.hidden_channels_d = cfg.get("hidden_channels_d", [32, 64, 128])

        self.seq_len = None
        self.n_features = None
        self.data_min = None
```

```
        self.data_max = None
        self.data_mean = None
        self.data_std = None

        self.G = None
        self.D = None
        self.opt_G = None
        self.opt_D = None

    def _normalize_data(self, data, fit=True):
        if fit:
            self.data_min = np.min(data, axis=(0, 1), keepdims=True)
            self.data_max = np.max(data, axis=(0, 1), keepdims=True)
            self.data_mean = np.mean(data, axis=(0, 1), keepdims=True)
            self.data_std = np.std(data, axis=(0, 1), keepdims=True) + 1e-8

            print(f"[QuantGAN] Data range: [{data.min():.4f}, {data.max():.
↪4f}]")
            print(f"[QuantGAN] Mean: {self.data_mean.squeeze():.6f}, Std: {self.
↪data_std.squeeze():.6f}")

        data_range = self.data_max - self.data_min + 1e-8
        normalized = 2 * (data - self.data_min) / data_range - 1
        normalized = np.clip(normalized, -0.99, 0.99)

        if fit:
            print(f"[QuantGAN] Normalized range: [{normalized.min():.4f},␣
↪{normalized.max():.4f}]")

        return normalized

    def _denormalize_data(self, data):
        data_range = self.data_max - self.data_min
        return (data + 1) / 2 * data_range + self.data_min

    def _make_windows(self, series, seq_len):
        T = series.shape[0]
        if T < seq_len:
            raise ValueError(f"Time series too short ({T}) for␣
↪seq_len={seq_len}")

        windows = []
        for i in range(T - seq_len + 1):
            windows.append(series[i:i+seq_len])

        return np.array(windows)
```

```python
    def _build_models(self, seq_len, n_features):
        self.seq_len = seq_len
        self.n_features = n_features

        self.G = QuantGAN_Generator(
            self.noise_dim, seq_len, n_features, self.hidden_channels_g
        ).to(self.device)

        self.D = QuantGAN_Discriminator(
            seq_len, n_features, self.hidden_channels_d
        ).to(self.device)

        self.opt_G = optim.Adam(self.G.parameters(), lr=self.lr_g, betas=(0.0,␣
↪0.9))
        self.opt_D = optim.Adam(self.D.parameters(), lr=self.lr_d, betas=(0.0,␣
↪0.9))

        self._init_weights(self.G)
        self._init_weights(self.D)

        print(f"[QuantGAN] Models built - G params: {sum(p.numel() for p in␣
↪self.G.parameters()):,}")
        print(f"[QuantGAN] Models built - D params: {sum(p.numel() for p in␣
↪self.D.parameters()):,}")

    def _init_weights(self, m):
        if isinstance(m, (nn.Conv1d, nn.ConvTranspose1d, nn.Linear)):
            nn.init.normal_(m.weight, 0.0, 0.02)
            if m.bias is not None:
                nn.init.constant_(m.bias, 0)
        elif isinstance(m, (nn.BatchNorm1d, nn.LayerNorm)):
            if hasattr(m, 'weight') and m.weight is not None:
                nn.init.normal_(m.weight, 1.0, 0.02)
            if hasattr(m, 'bias') and m.bias is not None:
                nn.init.constant_(m.bias, 0)

    def _gradient_penalty(self, real, fake):
        batch_size = real.size(0)
        alpha = torch.rand(batch_size, 1, 1, device=self.device)

        interp = (alpha * real + (1 - alpha) * fake).requires_grad_(True)
        d_interp = self.D(interp)

        gradients = torch.autograd.grad(
            outputs=d_interp,
            inputs=interp,
            grad_outputs=torch.ones_like(d_interp),
```

```
            create_graph=True,
            retain_graph=True,
            only_inputs=True
        )[0]

        gradients = gradients.reshape(batch_size, -1)
        grad_norm = gradients.norm(2, dim=1)
        penalty = torch.mean((grad_norm - 1) ** 2)

        return penalty

    def train(self, train_data):
        data = np.asarray(train_data, dtype=np.float32)

        if data.ndim == 1:
            data = data[:, None]

        if np.isnan(data).any():
            nan_count = np.isnan(data).sum()
            print(f"[QuantGAN] WARNING: Found {nan_count} NaN values, handling..
↪.")
            data_df = pd.DataFrame(data)
            data_df = data_df.fillna(method='ffill').fillna(method='bfill').
↪fillna(0.0)
            data = data_df.values
            print(f"[QuantGAN] NaN handling complete")

        T, n_features = data.shape

        max_seq_len = (T // 4) * 4
        if max_seq_len < 16:
            raise ValueError(f"Time series too short ({T}) for QuantGAN")

        if max_seq_len >= 256:
            seq_len = 256
        elif max_seq_len >= 128:
            seq_len = 128
        elif max_seq_len >= 64:
            seq_len = 64
        elif max_seq_len >= 32:
            seq_len = 32
        else:
            seq_len = max_seq_len

        print(f"[QuantGAN] Creating windows: T={T}, seq_len={seq_len},␣
↪n_features={n_features}")
```

```
        data = self._make_windows(data, seq_len)
        data = self._normalize_data(data, fit=True)

        n_windows = data.shape[0]
        print(f"[QuantGAN] Created {n_windows} windows")

        if self.G is None:
            self._build_models(seq_len, n_features)

        dataset = TensorDataset(torch.tensor(data, dtype=torch.float32))
        loader = DataLoader(dataset, self.batch_size, shuffle=True,␣
↪drop_last=True)

        print(f"[QuantGAN] Starting training with {len(loader)} batches per␣
↪epoch")

        for epoch in range(self.epochs):
            d_losses = []
            g_losses = []
            gp_losses = []

            for batch_idx, (real,) in enumerate(loader):
                real = real.to(self.device)
                batch_size = real.size(0)

                for _ in range(self.n_critic):
                    self.opt_D.zero_grad()

                    z = torch.randn(batch_size, self.noise_dim, device=self.
↪device)

                    with torch.no_grad():
                        fake = self.G(z)

                    d_real = self.D(real)
                    d_fake = self.D(fake)

                    d_loss = d_fake.mean() - d_real.mean()
                    gp = self._gradient_penalty(real, fake)
                    d_total = d_loss + self.lambda_gp * gp

                    if not (torch.isnan(d_total) or torch.isinf(d_total) or␣
↪abs(d_total.item()) > 1e6):
                        d_total.backward()
                        torch.nn.utils.clip_grad_norm_(self.D.parameters(),␣
↪max_norm=1.0)
                        self.opt_D.step()
```

```
                d_losses.append(d_total.item())
                gp_losses.append(gp.item())

                self.opt_G.zero_grad()
                z = torch.randn(batch_size, self.noise_dim, device=self.device)
                fake = self.G(z)
                g_loss = -self.D(fake).mean()

                if not (torch.isnan(g_loss) or torch.isinf(g_loss) or␣
↪abs(g_loss.item()) > 1e6):
                    g_loss.backward()
                    torch.nn.utils.clip_grad_norm_(self.G.parameters(),␣
↪max_norm=1.0)
                    self.opt_G.step()
                    g_losses.append(g_loss.item())

            if len(d_losses) > 0 and len(g_losses) > 0:
                avg_d = np.mean(d_losses)
                avg_g = np.mean(g_losses)
                avg_gp = np.mean(gp_losses)

                if (epoch + 1) % 5 == 0 or epoch == 0:
                    print(f"[QuantGAN] Epoch {epoch+1}/{self.epochs} | D={avg_d:
↪.4f} | G={avg_g:.4f} | GP={avg_gp:.4f}")

    def validate(self, val_data):
        data = np.asarray(val_data, dtype=np.float32)

        if data.ndim == 1:
            data = data[:, None]

        if np.isnan(data).any():
            data_df = pd.DataFrame(data)
            data_df = data_df.fillna(method='ffill').fillna(method='bfill').
↪fillna(0.0)
            data = data_df.values

        if self.seq_len is None:
            raise RuntimeError("Model not trained yet")

        T = data.shape[0]

        if T < self.seq_len:
            temp_seq_len = (T // 4) * 4
            if temp_seq_len < 16:
                return 0.0
```

```
            data = self._make_windows(data, temp_seq_len)
            data = self._normalize_data(data, fit=False)

            n_windows, _, n_features = data.shape
            padded_data = np.zeros((n_windows, self.seq_len, n_features),␣
↪dtype=np.float32)
            padded_data[:, :temp_seq_len, :] = data
            data = padded_data
        else:
            data = self._make_windows(data, self.seq_len)
            data = self._normalize_data(data, fit=False)

        real = torch.tensor(data, dtype=torch.float32).to(self.device)

        self.G.eval()
        self.D.eval()

        with torch.no_grad():
            batch_size = min(self.batch_size, real.size(0))
            z = torch.randn(batch_size, self.noise_dim, device=self.device)
            fake = self.G(z)
            score = (self.D(real[:batch_size]).mean() - self.D(fake).mean()).
↪item()

        self.G.train()
        self.D.train()

        return score

    def generate(self, n_samples):
        if self.G is None:
            raise RuntimeError("Model not trained yet")

        self.G.eval()

        n_windows = max(1, (n_samples + self.seq_len - 1) // self.seq_len)

        out = []
        with torch.no_grad():
            for i in range(0, n_windows, self.batch_size):
                b = min(self.batch_size, n_windows - i)
                z = torch.randn(b, self.noise_dim, device=self.device)
                fake_windows = self.G(z).cpu().numpy()
                out.append(fake_windows)

        self.G.train()
```

```
        windows = np.concatenate(out, axis=0)

        windows = self._denormalize_data(windows)
        reconstructed = windows.reshape(-1, self.n_features)
        reconstructed = reconstructed[:n_samples]

        if reconstructed.shape[0] < n_samples:
            padding = np.repeat(reconstructed[-1:], n_samples - reconstructed.
↪shape[0], axis=0)
            reconstructed = np.vstack([reconstructed, padding])

        return reconstructed
```

0.0.4 FinGAN

[6]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import pandas as pd
from torch.utils.data import DataLoader, TensorDataset

# -------------------------------
# Residual Block
# -------------------------------
class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv1d(channels, channels, 3, padding=1),
            nn.BatchNorm1d(channels),
            nn.LeakyReLU(0.2),
            nn.Conv1d(channels, channels, 3, padding=1),
            nn.BatchNorm1d(channels)
        )
        self.act = nn.LeakyReLU(0.2)

    def forward(self, x):
        return self.act(x + self.block(x))

# -------------------------------
# Generator (Deconv CNN)
# -------------------------------
class FinGAN_Generator(nn.Module):
    def __init__(self, noise_dim, seq_len, n_features, base_channels=64):
        super().__init__()
        assert seq_len % 8 == 0, "seq_len must be divisible by 8"
```

```
        self.start_len = seq_len // 8

        self.fc = nn.Sequential(
            nn.Linear(noise_dim, base_channels * self.start_len),
            nn.BatchNorm1d(base_channels * self.start_len),
            nn.LeakyReLU(0.2)
        )

        self.net = nn.Sequential(
            nn.ConvTranspose1d(base_channels, base_channels // 2, 4, 2, 1),
            nn.BatchNorm1d(base_channels // 2),
            nn.LeakyReLU(0.2),
            nn.ConvTranspose1d(base_channels // 2, base_channels // 4, 4, 2, 1),
            nn.BatchNorm1d(base_channels // 4),
            nn.LeakyReLU(0.2),
            nn.ConvTranspose1d(base_channels // 4, n_features, 4, 2, 1),
            nn.Tanh()
        )

    def forward(self, z):
        x = self.fc(z)
        x = x.view(z.size(0), -1, self.start_len)
        x = self.net(x)
        return x.permute(0, 2, 1)

# -------------------------------
# Critic (CNN)
# -------------------------------
class FinGAN_Critic(nn.Module):
    def __init__(self, seq_len, n_features, base_channels=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv1d(n_features, base_channels, 5, padding=2),
            nn.LayerNorm([base_channels, seq_len]),
            nn.LeakyReLU(0.2),
            nn.Conv1d(base_channels, base_channels * 2, 4, stride=2, padding=1),
            nn.LayerNorm([base_channels * 2, seq_len // 2]),
            nn.LeakyReLU(0.2),
            nn.Conv1d(base_channels * 2, base_channels * 4, 4, stride=2,␣
 ↪padding=1),
            nn.LayerNorm([base_channels * 4, seq_len // 4]),
            nn.LeakyReLU(0.2),
            nn.Flatten(),
            nn.Linear((seq_len // 4) * base_channels * 4, 128),
            nn.LeakyReLU(0.2),
            nn.Linear(128, 1)
```

```
        )

    def forward(self, x):
        return self.net(x.permute(0, 2, 1))

# -------------------------------
# FIN-GAN (WGAN-GP) - inherits from BaseGAN
# -------------------------------
class FinGAN(BaseGAN):
    def __init__(self, config=None):
        super().__init__(config or {})
        cfg = self.config

        self.device = torch.device("cuda" if torch.cuda.is_available() else␣
 ↪"cpu")

        self.noise_dim = cfg.get("noise_dim", 100)
        self.lr_g = cfg.get("lr_g", 1e-4)
        self.lr_d = cfg.get("lr_d", 1e-4)
        self.batch_size = cfg.get("batch_size", 64)
        self.epochs = cfg.get("epochs", 50)
        self.lambda_gp = cfg.get("lambda_gp", 10.0)
        self.n_critic = cfg.get("n_critic", 3)
        self.base_channels = cfg.get("base_channels", 32)

        self.seq_len = None
        self.n_features = None
        self.data_min = None
        self.data_max = None
        self.data_mean = None
        self.data_std = None

        self.G = None
        self.D = None
        self.opt_G = None
        self.opt_D = None

    def _normalize_data(self, data, fit=True):
        if fit:
            self.data_min = np.min(data, axis=(0, 1), keepdims=True)
            self.data_max = np.max(data, axis=(0, 1), keepdims=True)
            self.data_mean = np.mean(data, axis=(0, 1), keepdims=True)
            self.data_std = np.std(data, axis=(0, 1), keepdims=True) + 1e-8

            print(f"[FinGAN] Data range: [{data.min():.4f}, {data.max():.4f}]")
            print(f"[FinGAN] Mean: {self.data_mean.squeeze():.6f}, Std: {self.
 ↪data_std.squeeze():.6f}")
```

```
        data_range = self.data_max - self.data_min + 1e-8
        normalized = 2 * (data - self.data_min) / data_range - 1
        normalized = np.clip(normalized, -0.99, 0.99)

        if fit:
            print(f"[FinGAN] Normalized range: [{normalized.min():.4f},␣
↪{normalized.max():.4f}]")

        return normalized

    def _denormalize_data(self, data):
        data_range = self.data_max - self.data_min
        return (data + 1) / 2 * data_range + self.data_min

    def _make_windows(self, series, seq_len):
        T = series.shape[0]
        if T < seq_len:
            raise ValueError(f"Time series too short ({T}) for␣
↪seq_len={seq_len}")

        windows = []
        for i in range(T - seq_len + 1):
            windows.append(series[i:i+seq_len])

        return np.array(windows)

    def _adjust_seq_len(self, data, factor=8):
        n, seq_len, n_features = data.shape
        target_len = (seq_len // factor) * factor
        if target_len != seq_len:
            print(f"[FinGAN] Truncated seq_len {seq_len} → {target_len}")
        return data[:, :target_len, :]

    def _build_models(self, seq_len, n_features):
        self.seq_len = seq_len
        self.n_features = n_features

        self.G = FinGAN_Generator(
            self.noise_dim, seq_len, n_features, self.base_channels
        ).to(self.device)

        self.D = FinGAN_Critic(
            seq_len, n_features, self.base_channels
        ).to(self.device)
```

```
        self.opt_G = optim.Adam(self.G.parameters(), lr=self.lr_g, betas=(0.0,␣
↪0.9))
        self.opt_D = optim.Adam(self.D.parameters(), lr=self.lr_d, betas=(0.0,␣
↪0.9))

        self._init_weights(self.G)
        self._init_weights(self.D)

        print(f"[FinGAN] Models built - G params: {sum(p.numel() for p in self.
↪G.parameters()):,}")
        print(f"[FinGAN] Models built - D params: {sum(p.numel() for p in self.
↪D.parameters()):,}")

    def _init_weights(self, m):
        if isinstance(m, (nn.Conv1d, nn.ConvTranspose1d, nn.Linear)):
            nn.init.normal_(m.weight, 0.0, 0.02)
            if m.bias is not None:
                nn.init.constant_(m.bias, 0)
        elif isinstance(m, (nn.BatchNorm1d, nn.LayerNorm)):
            if hasattr(m, 'weight') and m.weight is not None:
                nn.init.normal_(m.weight, 1.0, 0.02)
            if hasattr(m, 'bias') and m.bias is not None:
                nn.init.constant_(m.bias, 0)

    def _gradient_penalty(self, real, fake):
        batch_size = real.size(0)
        alpha = torch.rand(batch_size, 1, 1, device=self.device)

        interp = (alpha * real + (1 - alpha) * fake).requires_grad_(True)
        d_interp = self.D(interp)

        gradients = torch.autograd.grad(
            outputs=d_interp,
            inputs=interp,
            grad_outputs=torch.ones_like(d_interp),
            create_graph=True,
            retain_graph=True,
            only_inputs=True
        )[0]

        gradients = gradients.reshape(batch_size, -1)
        grad_norm = gradients.norm(2, dim=1)
        penalty = torch.mean((grad_norm - 1) ** 2)

        return penalty

    def train(self, train_data):
```

```
        data = np.asarray(train_data, dtype=np.float32)

        if data.ndim == 1:
            data = data[:, None]

        if np.isnan(data).any():
            nan_count = np.isnan(data).sum()
            print(f"[WARNING] Found {nan_count} NaN values, handling...")
            data_df = pd.DataFrame(data)
            data_df = data_df.fillna(method='ffill').fillna(method='bfill').
↪fillna(0.0)
            data = data_df.values
            print(f"[FinGAN] NaN handling complete")

        T, n_features = data.shape

        max_seq_len = (T // 8) * 8
        if max_seq_len < 8:
            raise ValueError(f"Time series too short ({T}) for FIN-GAN")

        if max_seq_len >= 256:
            seq_len = 256
        elif max_seq_len >= 128:
            seq_len = 128
        elif max_seq_len >= 64:
            seq_len = 64
        elif max_seq_len >= 32:
            seq_len = 32
        elif max_seq_len >= 16:
            seq_len = 16
        else:
            seq_len = max_seq_len

        print(f"[FinGAN] Creating windows: T={T}, seq_len={seq_len},␣
↪n_features={n_features}")

        data = self._make_windows(data, seq_len)

        if np.isnan(data).any():
            raise ValueError("NaN after windowing")

        data = self._normalize_data(data, fit=True)

        if np.isnan(data).any():
            raise ValueError("NaN after normalization")

        n_windows = data.shape[0]
```

```
        print(f"[FinGAN] Created {n_windows} windows")

        if self.G is None:
            self._build_models(seq_len, n_features)

        dataset = TensorDataset(torch.tensor(data, dtype=torch.float32))
        loader = DataLoader(dataset, self.batch_size, shuffle=True,␣
↪drop_last=True)

        print(f"[FinGAN] Starting training with {len(loader)} batches per␣
↪epoch")

        for epoch in range(self.epochs):
            d_losses = []
            g_losses = []
            gp_losses = []

            for batch_idx, (real,) in enumerate(loader):
                real = real.to(self.device)
                batch_size = real.size(0)

                if torch.isnan(real).any():
                    print(f"[ERROR] NaN in batch {batch_idx}")
                    continue

                for critic_iter in range(self.n_critic):
                    self.opt_D.zero_grad()

                    z = torch.randn(batch_size, self.noise_dim, device=self.
↪device)

                    with torch.no_grad():
                        fake = self.G(z)

                    d_real = self.D(real)
                    d_fake = self.D(fake)

                    d_loss = d_fake.mean() - d_real.mean()
                    gp = self._gradient_penalty(real, fake)
                    d_total = d_loss + self.lambda_gp * gp

                    if torch.isnan(d_total) or torch.isinf(d_total) or␣
↪abs(d_total.item()) > 1e6:
                        print(f"[ERROR] Extreme D loss at epoch {epoch+1},␣
↪batch {batch_idx}")
                        break
```

```
                    d_total.backward()
                    torch.nn.utils.clip_grad_norm_(self.D.parameters(),␣
↪max_norm=0.5)
                    self.opt_D.step()

                d_losses.append(d_total.item())
                gp_losses.append(gp.item())

                self.opt_G.zero_grad()
                z = torch.randn(batch_size, self.noise_dim, device=self.device)
                fake = self.G(z)
                g_loss = -self.D(fake).mean()

                if torch.isnan(g_loss) or torch.isinf(g_loss) or abs(g_loss.
↪item()) > 1e6:
                    print(f"[ERROR] Extreme G loss")
                    continue

                g_loss.backward()
                torch.nn.utils.clip_grad_norm_(self.G.parameters(), max_norm=0.
↪5)
                self.opt_G.step()

                g_losses.append(g_loss.item())

            if len(d_losses) == 0 or len(g_losses) == 0:
                print(f"[ERROR] No valid losses in epoch {epoch+1}")
                break

            avg_d = np.mean(d_losses)
            avg_g = np.mean(g_losses)
            avg_gp = np.mean(gp_losses)

            if (epoch + 1) % 5 == 0 or epoch == 0:
                print(f"[FinGAN] Epoch {epoch+1}/{self.epochs} | D={avg_d:.4f}␣
↪| G={avg_g:.4f} | GP={avg_gp:.4f}")

    def validate(self, val_data):
        data = np.asarray(val_data, dtype=np.float32)

        if data.ndim == 1:
            data = data[:, None]

        if np.isnan(data).any():
            data_df = pd.DataFrame(data)
            data_df = data_df.fillna(method='ffill').fillna(method='bfill').
↪fillna(0.0)
```

```
            data = data_df.values

        if self.seq_len is None:
            raise RuntimeError("Model not trained yet")

        T = data.shape[0]

        if T < self.seq_len:
            temp_seq_len = (T // 8) * 8
            if temp_seq_len < 8:
                return 0.0

            data = self._make_windows(data, temp_seq_len)
            data = self._normalize_data(data, fit=False)

            n_windows, _, n_features = data.shape
            padded_data = np.zeros((n_windows, self.seq_len, n_features),␣
↪dtype=np.float32)
            padded_data[:, :temp_seq_len, :] = data
            data = padded_data
        else:
            data = self._make_windows(data, self.seq_len)
            data = self._normalize_data(data, fit=False)

        data = self._adjust_seq_len(data, factor=8)
        real = torch.tensor(data, dtype=torch.float32).to(self.device)

        self.G.eval()
        self.D.eval()

        with torch.no_grad():
            batch_size = min(self.batch_size, real.size(0))
            z = torch.randn(batch_size, self.noise_dim, device=self.device)
            fake = self.G(z)
            score = (self.D(real[:batch_size]).mean() - self.D(fake).mean()).
↪item()

        self.G.train()
        self.D.train()

        return score

    def generate(self, n_samples):
        """Generate n_samples timesteps (not windows)"""
        if self.G is None:
            raise RuntimeError("Model not trained yet")
```

```
        self.G.eval()

        # Generate enough windows to cover n_samples timesteps
        n_windows = max(1, (n_samples + self.seq_len - 1) // self.seq_len)

        out = []
        with torch.no_grad():
            for i in range(0, n_windows, self.batch_size):
                b = min(self.batch_size, n_windows - i)
                z = torch.randn(b, self.noise_dim, device=self.device)
                fake_windows = self.G(z).cpu().numpy()
                out.append(fake_windows)

        self.G.train()
        windows = np.concatenate(out, axis=0)

        # Denormalize
        windows = self._denormalize_data(windows)

        # Flatten windows into continuous time series
        reconstructed = windows.reshape(-1, self.n_features)

        # Trim to exact n_samples
        reconstructed = reconstructed[:n_samples]

        # Pad if needed
        if reconstructed.shape[0] < n_samples:
            padding = np.repeat(reconstructed[-1:], n_samples - reconstructed.
↪shape[0], axis=0)
            reconstructed = np.vstack([reconstructed, padding])

        return reconstructed
```

0.0.5 Financial Metrics

[7]:
```
import numpy as np
import pandas as pd
from scipy import stats
from scipy.stats import ks_2samp, anderson_ksamp
from sklearn.metrics import mean_squared_error, mean_absolute_error
import matplotlib.pyplot as plt
import seaborn as sns

class FinancialMetrics:
    """
    Compute stylized facts and evaluation metrics for financial time series.
    Based on standard financial econometrics literature.
```

```
    """

    def __init__(self, real_data, synthetic_data):
        """
        Args:
            real_data: (n_samples,) or (n_samples, 1) array
            synthetic_data: (n_samples,) or (n_samples, 1) array
        """
        self.real = np.asarray(real_data).flatten()
        self.synthetic = np.asarray(synthetic_data).flatten()

    # ========================================
    # 1. DISTRIBUTION METRICS
    # ========================================

    def compute_moments(self):
        """Compute first 4 statistical moments"""
        moments_real = {
            'mean': np.mean(self.real),
            'std': np.std(self.real),
            'skewness': stats.skew(self.real),
            'kurtosis': stats.kurtosis(self.real),  # Excess kurtosis
        }

        moments_syn = {
            'mean': np.mean(self.synthetic),
            'std': np.std(self.synthetic),
            'skewness': stats.skew(self.synthetic),
            'kurtosis': stats.kurtosis(self.synthetic),
        }

        # Compute differences
        diff = {
            'mean_diff': abs(moments_real['mean'] - moments_syn['mean']),
            'std_diff': abs(moments_real['std'] - moments_syn['std']),
            'skewness_diff': abs(moments_real['skewness'] -␣
↪moments_syn['skewness']),
            'kurtosis_diff': abs(moments_real['kurtosis'] -␣
↪moments_syn['kurtosis']),
        }

        return {
            'real': moments_real,
            'synthetic': moments_syn,
            'differences': diff
        }
```

```python
    def compute_distribution_tests(self):
        """Statistical tests for distribution similarity"""
        # Kolmogorov-Smirnov test
        ks_stat, ks_pvalue = ks_2samp(self.real, self.synthetic)

        # Anderson-Darling test (more sensitive to tails)
        ad_result = anderson_ksamp([self.real, self.synthetic])

        # Wasserstein distance (Earth Mover's Distance)
        wasserstein = stats.wasserstein_distance(self.real, self.synthetic)

        return {
            'ks_statistic': ks_stat,
            'ks_pvalue': ks_pvalue,
            'anderson_darling_statistic': ad_result.statistic,
            'anderson_darling_pvalue': ad_result.pvalue if hasattr(ad_result,␣
↪'pvalue') else ad_result.significance_level,
            'wasserstein_distance': wasserstein
        }

    # ========================================
    # 2. AUTOCORRELATION METRICS
    # ========================================

    def compute_autocorrelation(self, max_lag=20):
        """
        Compute autocorrelation for returns, absolute returns, and squared␣
↪returns.
        This captures volatility clustering.
        """
        def acf(x, max_lag):
            """Compute autocorrelation function"""
            x = x - np.mean(x)
            c0 = np.dot(x, x) / len(x)
            acf_values = np.array([np.dot(x[:-lag], x[lag:]) / len(x) / c0
                                   if lag > 0 else 1.0
                                   for lag in range(max_lag + 1)])
            return acf_values

        # ACF of returns (should be close to 0)
        acf_real = acf(self.real, max_lag)
        acf_syn = acf(self.synthetic, max_lag)

        # ACF of absolute returns (volatility clustering)
        acf_abs_real = acf(np.abs(self.real), max_lag)
        acf_abs_syn = acf(np.abs(self.synthetic), max_lag)
```

```
        # ACF of squared returns (volatility clustering)
        acf_sq_real = acf(self.real**2, max_lag)
        acf_sq_syn = acf(self.synthetic**2, max_lag)

        # Compute differences (MAE across lags)
        acf_diff = np.mean(np.abs(acf_real[1:] - acf_syn[1:]))
        acf_abs_diff = np.mean(np.abs(acf_abs_real[1:] - acf_abs_syn[1:]))
        acf_sq_diff = np.mean(np.abs(acf_sq_real[1:] - acf_sq_syn[1:]))

        return {
            'acf_returns': {'real': acf_real, 'synthetic': acf_syn, 'mae':␣
↪acf_diff},
            'acf_absolute': {'real': acf_abs_real, 'synthetic': acf_abs_syn,␣
↪'mae': acf_abs_diff},
            'acf_squared': {'real': acf_sq_real, 'synthetic': acf_sq_syn, 'mae':
↪ acf_sq_diff},
        }

    def ljung_box_test(self, lags=20):
        """
        Ljung-Box test for autocorrelation.
        Tests if any group of autocorrelations is different from zero.
        """
        from statsmodels.stats.diagnostic import acorr_ljungbox

        # Test on returns
        lb_real = acorr_ljungbox(self.real, lags=lags, return_df=False)
        lb_syn = acorr_ljungbox(self.synthetic, lags=lags, return_df=False)

        # Test on absolute returns (volatility)
        lb_abs_real = acorr_ljungbox(np.abs(self.real), lags=lags,␣
↪return_df=False)
        lb_abs_syn = acorr_ljungbox(np.abs(self.synthetic), lags=lags,␣
↪return_df=False)

        return {
            'returns_real_pvalue': lb_real[1][-1],  # Last lag p-value
            'returns_synthetic_pvalue': lb_syn[1][-1],
            'absolute_real_pvalue': lb_abs_real[1][-1],
            'absolute_synthetic_pvalue': lb_abs_syn[1][-1],
        }

    # ========================================
    # 3. TAIL BEHAVIOR (FAT TAILS)
    # ========================================

    def compute_tail_index(self, q=0.95):
```

```
        """
        Estimate tail index using Hill estimator.
        Lower values indicate heavier tails (more extreme events).
        """
        def hill_estimator(data, q=0.95):
            """Hill estimator for tail index"""
            sorted_data = np.sort(np.abs(data))
            threshold_idx = int(len(sorted_data) * q)
            tail_data = sorted_data[threshold_idx:]

            if len(tail_data) < 2:
                return np.nan

            log_ratios = np.log(tail_data / tail_data[0])
            alpha = 1.0 / np.mean(log_ratios[1:])
            return alpha

        tail_real = hill_estimator(self.real, q)
        tail_syn = hill_estimator(self.synthetic, q)

        return {
            'tail_index_real': tail_real,
            'tail_index_synthetic': tail_syn,
            'tail_index_diff': abs(tail_real - tail_syn) if not np.
↪isnan(tail_real) else np.nan
        }

    def compute_extreme_events(self, threshold_std=2.0):
        """
        Count extreme events (beyond threshold standard deviations).
        Financial data typically has more extreme events than normal␣
↪distribution.
        """
        real_std = np.std(self.real)
        syn_std = np.std(self.synthetic)

        extreme_real = np.sum(np.abs(self.real) > threshold_std * real_std)
        extreme_syn = np.sum(np.abs(self.synthetic) > threshold_std * syn_std)

        # Normalize by sample size
        extreme_real_pct = 100 * extreme_real / len(self.real)
        extreme_syn_pct = 100 * extreme_syn / len(self.synthetic)

        # For normal distribution, expect ~5% beyond 2 , ~0.3% beyond 3
        expected_pct = 100 * (1 - stats.norm.cdf(threshold_std) + stats.norm.
↪cdf(-threshold_std))
```

```
        return {
            'extreme_real_count': extreme_real,
            'extreme_synthetic_count': extreme_syn,
            'extreme_real_pct': extreme_real_pct,
            'extreme_synthetic_pct': extreme_syn_pct,
            'expected_normal_pct': expected_pct,
            'diff_from_expected_real': extreme_real_pct - expected_pct,
            'diff_from_expected_synthetic': extreme_syn_pct - expected_pct,
        }

    # ========================================
    # 4. BASIC ERROR METRICS
    # ========================================

    def compute_basic_metrics(self):
        """Standard ML metrics"""
        return {
            'mse': mean_squared_error(self.real, self.synthetic),
            'rmse': np.sqrt(mean_squared_error(self.real, self.synthetic)),
            'mae': mean_absolute_error(self.real, self.synthetic),
            'mape': np.mean(np.abs((self.real - self.synthetic) / (self.real +␣
↪1e-8))) * 100,
        }

    # ========================================
    # 5. COMPREHENSIVE REPORT
    # ========================================

    def compute_all_metrics(self, max_lag=20):
        """Compute all metrics and return as dictionary"""
        print("Computing comprehensive financial metrics...")

        metrics = {}

        # Distribution metrics
        print("  - Distribution moments...")
        metrics['moments'] = self.compute_moments()

        print("  - Distribution tests...")
        metrics['distribution_tests'] = self.compute_distribution_tests()

        # Autocorrelation (volatility clustering)
        print("  - Autocorrelation analysis...")
        metrics['autocorrelation'] = self.compute_autocorrelation(max_lag)

        print("  - Ljung-Box test...")
        try:
```

```
            metrics['ljung_box'] = self.ljung_box_test(lags=min(max_lag,␣
↪len(self.real)//2))
        except Exception as e:
            print(f"    Warning: Ljung-Box test failed: {e}")
            metrics['ljung_box'] = None

        # Tail behavior
        print("  - Tail index estimation...")
        metrics['tail_behavior'] = self.compute_tail_index()

        print("  - Extreme events analysis...")
        metrics['extreme_events'] = self.compute_extreme_events(threshold_std=2.
↪0)

        # Basic metrics
        print("  - Basic error metrics...")
        metrics['basic_metrics'] = self.compute_basic_metrics()

        return metrics

    def print_summary(self, metrics=None):
        """Print a formatted summary of metrics"""
        if metrics is None:
            metrics = self.compute_all_metrics()

        print("\n" + "="*70)
        print(" FINANCIAL TIME SERIES EVALUATION SUMMARY")
        print("="*70)

        # Moments
        print("\n1. DISTRIBUTION MOMENTS:")
        print("-" * 70)
        moments = metrics['moments']
        for key in ['mean', 'std', 'skewness', 'kurtosis']:
            real_val = moments['real'][key]
            syn_val = moments['synthetic'][key]
            diff = moments['differences'][f'{key}_diff']
            print(f"  {key.upper():12s}: Real={real_val:8.4f} |␣
↪Synthetic={syn_val:8.4f} | Diff={diff:8.4f}")

        # Distribution tests
        print("\n2. DISTRIBUTION TESTS:")
        print("-" * 70)
        dist_tests = metrics['distribution_tests']
        print(f"  KS Test:        stat={dist_tests['ks_statistic']:.4f},␣
↪p={dist_tests['ks_pvalue']:.4f}")
```

```
        print(f"  Wasserstein:    distance={dist_tests['wasserstein_distance']:.
↪4f}")

        # Autocorrelation
        print("\n3. AUTOCORRELATION (Volatility Clustering):")
        print("-" * 70)
        acf = metrics['autocorrelation']
        print(f"  Returns ACF MAE:         {acf['acf_returns']['mae']:.4f}")
        print(f"  Absolute Returns ACF MAE: {acf['acf_absolute']['mae']:.4f}")
        print(f"  Squared Returns ACF MAE:  {acf['acf_squared']['mae']:.4f}")

        # Tail behavior
        print("\n4. TAIL BEHAVIOR (Fat Tails):")
        print("-" * 70)
        tail = metrics['tail_behavior']
        print(f"  Hill Estimator (Real):      {tail['tail_index_real']:.4f}")
        print(f"  Hill Estimator (Synthetic): {tail['tail_index_synthetic']:.
↪4f}")

        extreme = metrics['extreme_events']
        print(f"  Extreme Events (Real):      {extreme['extreme_real_pct']:.
↪2f}%")
        print(f"  Extreme Events (Synthetic): {extreme['extreme_synthetic_pct']:
↪.2f}%")
        print(f"  Expected (Normal):          {extreme['expected_normal_pct']:.
↪2f}%")

        # Basic metrics
        print("\n5. BASIC ERROR METRICS:")
        print("-" * 70)
        basic = metrics['basic_metrics']
        print(f"  MSE:  {basic['mse']:.6f}")
        print(f"  RMSE: {basic['rmse']:.6f}")
        print(f"  MAE:  {basic['mae']:.6f}")

        print("\n" + "="*70)

        return metrics


# ========================================
# CONVENIENCE FUNCTION
# ========================================

def evaluate_gan(real_data, synthetic_data, max_lag=20, print_summary=True):
    """
    Convenience function to evaluate GAN-generated financial time series.
```

```
    Args:
        real_data: Real financial returns
        synthetic_data: GAN-generated returns
        max_lag: Maximum lag for autocorrelation
        print_summary: Whether to print formatted summary

    Returns:
        Dictionary of all computed metrics
    """
    evaluator = FinancialMetrics(real_data, synthetic_data)
    metrics = evaluator.compute_all_metrics(max_lag=max_lag)

    if print_summary:
        evaluator.print_summary(metrics)

    return metrics


# ========================================
# BATCH EVALUATION FOR MULTIPLE MODELS
# ========================================

def evaluate_multiple_models(real_data, models_dict, max_lag=20):
    """
    Evaluate multiple GAN models against real data.

    Args:
        real_data: Real financial returns (n_samples,)
        models_dict: Dictionary {model_name: synthetic_data}
        max_lag: Maximum lag for autocorrelation

    Returns:
        DataFrame with comparison of all models
    """
    results = []

    for model_name, synthetic_data in models_dict.items():
        print(f"\n{'='*70}")
        print(f"Evaluating: {model_name}")
        print(f"{'='*70}")

        evaluator = FinancialMetrics(real_data, synthetic_data)
        metrics = evaluator.compute_all_metrics(max_lag=max_lag)

        # Flatten metrics for DataFrame
        row = {
```

```
            'model': model_name,
            'mean_diff': metrics['moments']['differences']['mean_diff'],
            'std_diff': metrics['moments']['differences']['std_diff'],
            'skewness_diff': metrics['moments']['differences']['skewness_diff'],
            'kurtosis_diff': metrics['moments']['differences']['kurtosis_diff'],
            'ks_statistic': metrics['distribution_tests']['ks_statistic'],
            'ks_pvalue': metrics['distribution_tests']['ks_pvalue'],
            'wasserstein':␣
↪metrics['distribution_tests']['wasserstein_distance'],
            'acf_returns_mae': metrics['autocorrelation']['acf_returns']['mae'],
            'acf_absolute_mae':␣
↪metrics['autocorrelation']['acf_absolute']['mae'],
            'acf_squared_mae': metrics['autocorrelation']['acf_squared']['mae'],
            'tail_index_diff': metrics['tail_behavior']['tail_index_diff'],
            'extreme_events_diff':␣
↪abs(metrics['extreme_events']['extreme_real_pct'] -
                                                    ␣
↪metrics['extreme_events']['extreme_synthetic_pct']),
            'mse': metrics['basic_metrics']['mse'],
            'mae': metrics['basic_metrics']['mae'],
        }

        results.append(row)

    df = pd.DataFrame(results)

    # Add ranking for each metric (lower is better)
    metrics_to_rank = [col for col in df.columns if col != 'model']
    for col in metrics_to_rank:
        df[f'{col}_rank'] = df[col].rank()

    # Compute average rank
    rank_cols = [col for col in df.columns if col.endswith('_rank')]
    df['avg_rank'] = df[rank_cols].mean(axis=1)

    return df.sort_values('avg_rank')
```

0.0.6 Financial Plots

[8]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# Set style
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (15, 10)
```

```python
plt.rcParams['font.size'] = 10

def plot_stylized_facts_comparison(real_data, synthetic_dict, max_lag=50,␣
↪save_path=None):
    """
    Create comprehensive stylized facts comparison plots.

    Args:
        real_data: Real returns (n_samples,)
        synthetic_dict: Dictionary {model_name: synthetic_data}
        max_lag: Maximum lag for ACF
        save_path: Path to save figure (optional)
    """
    real = np.asarray(real_data).flatten()
    n_models = len(synthetic_dict)

    # LARGER FIGURE with more spacing
    fig = plt.figure(figsize=(20, 16))
    gs = fig.add_gridspec(4, 3, hspace=0.4, wspace=0.35,
                          left=0.08, right=0.98, top=0.94, bottom=0.06)

    colors = plt.cm.Set2(np.linspace(0, 1, n_models + 1))

    # ========================================
    # Row 1: Time Series
    # ========================================
    ax1 = fig.add_subplot(gs[0, :])
    ax1.plot(real[:500], label='Real', color='black', alpha=0.7, linewidth=1.5)
    for idx, (name, syn_data) in enumerate(synthetic_dict.items()):
        syn = np.asarray(syn_data).flatten()
        ax1.plot(syn[:500], label=name, color=colors[idx+1], alpha=0.6,␣
↪linewidth=1)
    ax1.set_title('Time Series (First 500 Points)', fontsize=16,␣
↪fontweight='bold', pad=15)
    ax1.set_xlabel('Time', fontsize=12)
    ax1.set_ylabel('Returns', fontsize=12)
    ax1.legend(loc='best', frameon=True, shadow=True, fontsize=11)
    ax1.grid(True, alpha=0.3)
    ax1.tick_params(labelsize=10)

    # ========================================
    # Row 2: Distributions
    # ========================================

    # Distribution (Histogram + KDE)
    ax2 = fig.add_subplot(gs[1, 0])
```

```
    ax2.hist(real, bins=50, density=True, alpha=0.5, color='black',␣
↪label='Real', edgecolor='black')
    for idx, (name, syn_data) in enumerate(synthetic_dict.items()):
        syn = np.asarray(syn_data).flatten()
        ax2.hist(syn, bins=50, density=True, alpha=0.3, color=colors[idx+1],␣
↪label=name)
    ax2.set_title('Return Distribution', fontsize=13, fontweight='bold', pad=10)
    ax2.set_xlabel('Returns', fontsize=11)
    ax2.set_ylabel('Density', fontsize=11)
    ax2.legend(loc='best', fontsize=9)
    ax2.grid(True, alpha=0.3)
    ax2.tick_params(labelsize=9)

    # Q-Q Plot against Normal
    ax3 = fig.add_subplot(gs[1, 1])
    stats.probplot(real, dist="norm", plot=ax3)
    ax3.set_title('Q-Q Plot (Real vs Normal)', fontsize=13, fontweight='bold',␣
↪pad=10)
    ax3.grid(True, alpha=0.3)
    ax3.tick_params(labelsize=9)

    # Log-scale distribution (Fat Tails)
    ax4 = fig.add_subplot(gs[1, 2])
    ax4.hist(real, bins=50, density=True, alpha=0.5, color='black',␣
↪label='Real', edgecolor='black')
    for idx, (name, syn_data) in enumerate(synthetic_dict.items()):
        syn = np.asarray(syn_data).flatten()
        ax4.hist(syn, bins=50, density=True, alpha=0.3, color=colors[idx+1],␣
↪label=name)
    ax4.set_yscale('log')
    ax4.set_title('Distribution (Log Scale - Fat Tails)', fontsize=13,␣
↪fontweight='bold', pad=10)
    ax4.set_xlabel('Returns', fontsize=11)
    ax4.set_ylabel('Log Density', fontsize=11)
    ax4.legend(loc='best', fontsize=9)
    ax4.grid(True, alpha=0.3)
    ax4.tick_params(labelsize=9)

    # ========================================
    # Row 3: Autocorrelation Functions
    # ========================================

    def compute_acf(x, max_lag):
        x = x - np.mean(x)
        c0 = np.dot(x, x) / len(x)
```

```
        return np.array([np.dot(x[:-lag], x[lag:]) / len(x) / c0 if lag > 0␣
↪else 1.0
                        for lag in range(max_lag + 1)])

    # ACF of Returns
    ax5 = fig.add_subplot(gs[2, 0])
    lags = np.arange(max_lag + 1)
    acf_real = compute_acf(real, max_lag)
    ax5.stem(lags, acf_real, linefmt='black', markerfmt='ko', label='Real',␣
↪basefmt=" ")
    for idx, (name, syn_data) in enumerate(synthetic_dict.items()):
        syn = np.asarray(syn_data).flatten()
        acf_syn = compute_acf(syn, max_lag)
        ax5.plot(lags, acf_syn, marker='o', alpha=0.6, color=colors[idx+1],␣
↪label=name, markersize=3)
    ax5.axhline(y=0, color='gray', linestyle='--', linewidth=0.8)
    ax5.axhline(y=1.96/np.sqrt(len(real)), color='red', linestyle='--',␣
↪linewidth=0.8, alpha=0.5)
    ax5.axhline(y=-1.96/np.sqrt(len(real)), color='red', linestyle='--',␣
↪linewidth=0.8, alpha=0.5)
    ax5.set_title('ACF of Returns', fontsize=13, fontweight='bold', pad=10)
    ax5.set_xlabel('Lag', fontsize=11)
    ax5.set_ylabel('Autocorrelation', fontsize=11)
    ax5.legend(loc='best', fontsize=9)
    ax5.grid(True, alpha=0.3)
    ax5.tick_params(labelsize=9)

    # ACF of Absolute Returns (Volatility Clustering)
    ax6 = fig.add_subplot(gs[2, 1])
    acf_abs_real = compute_acf(np.abs(real), max_lag)
    ax6.stem(lags, acf_abs_real, linefmt='black', markerfmt='ko', label='Real',␣
↪basefmt=" ")
    for idx, (name, syn_data) in enumerate(synthetic_dict.items()):
        syn = np.asarray(syn_data).flatten()
        acf_abs_syn = compute_acf(np.abs(syn), max_lag)
        ax6.plot(lags, acf_abs_syn, marker='o', alpha=0.6, color=colors[idx+1],␣
↪label=name, markersize=3)
    ax6.axhline(y=0, color='gray', linestyle='--', linewidth=0.8)
    ax6.set_title('ACF of |Returns| (Volatility Clustering)', fontsize=13,␣
↪fontweight='bold', pad=10)
    ax6.set_xlabel('Lag', fontsize=11)
    ax6.set_ylabel('Autocorrelation', fontsize=11)
    ax6.legend(loc='best', fontsize=9)
    ax6.grid(True, alpha=0.3)
    ax6.tick_params(labelsize=9)
```

```
    # ACF of Squared Returns
    ax7 = fig.add_subplot(gs[2, 2])
    acf_sq_real = compute_acf(real**2, max_lag)
    ax7.stem(lags, acf_sq_real, linefmt='black', markerfmt='ko', label='Real',␣
↪basefmt=" ")
    for idx, (name, syn_data) in enumerate(synthetic_dict.items()):
        syn = np.asarray(syn_data).flatten()
        acf_sq_syn = compute_acf(syn**2, max_lag)
        ax7.plot(lags, acf_sq_syn, marker='o', alpha=0.6, color=colors[idx+1],␣
↪label=name, markersize=3)
    ax7.axhline(y=0, color='gray', linestyle='--', linewidth=0.8)
    ax7.set_title('ACF of Returns² (Volatility Clustering)', fontsize=13,␣
↪fontweight='bold', pad=10)
    ax7.set_xlabel('Lag', fontsize=11)
    ax7.set_ylabel('Autocorrelation', fontsize=11)
    ax7.legend(loc='best', fontsize=9)
    ax7.grid(True, alpha=0.3)
    ax7.tick_params(labelsize=9)

    # ========================================
    # Row 4: Statistical Properties
    # ========================================

    # Box plots of moments
    ax8 = fig.add_subplot(gs[3, 0])
    moments_data = []
    labels = ['Real'] + list(synthetic_dict.keys())

    for data in [real] + list(synthetic_dict.values()):
        d = np.asarray(data).flatten()
        moments_data.append(d)

    bp = ax8.boxplot(moments_data, labels=labels, patch_artist=True,␣
↪showfliers=False)
    for patch, color in zip(bp['boxes'], [colors[0]] + list(colors[1:
↪n_models+1])):
        patch.set_facecolor(color)
        patch.set_alpha(0.6)
    ax8.set_title('Distribution Comparison (Box Plot)', fontsize=13,␣
↪fontweight='bold', pad=10)
    ax8.set_ylabel('Returns', fontsize=11)
    ax8.grid(True, alpha=0.3, axis='y')
    ax8.tick_params(labelsize=9, axis='x', rotation=25)

    # Statistical Moments Comparison
    ax9 = fig.add_subplot(gs[3, 1])
```

```
    moment_names = ['Mean', 'Std', 'Skewness', 'Kurtosis']
    x_pos = np.arange(len(moment_names))
    width = 0.7 / (n_models + 1)

    real_moments = [np.mean(real), np.std(real), stats.skew(real), stats.
↪kurtosis(real)]
    ax9.bar(x_pos - width*n_models/2, real_moments, width, label='Real',␣
↪color='black', alpha=0.7)

    for idx, (name, syn_data) in enumerate(synthetic_dict.items()):
        syn = np.asarray(syn_data).flatten()
        syn_moments = [np.mean(syn), np.std(syn), stats.skew(syn), stats.
↪kurtosis(syn)]
        ax9.bar(x_pos - width*n_models/2 + width*(idx+1), syn_moments, width,
                label=name, color=colors[idx+1], alpha=0.7)

    ax9.set_xticks(x_pos)
    ax9.set_xticklabels(moment_names, fontsize=10)
    ax9.set_title('Statistical Moments', fontsize=13, fontweight='bold', pad=10)
    ax9.set_ylabel('Value', fontsize=11)
    ax9.legend(loc='best', fontsize=9)
    ax9.grid(True, alpha=0.3, axis='y')
    ax9.axhline(y=0, color='black', linewidth=0.8)
    ax9.tick_params(labelsize=9)

    # Volatility Comparison
    ax10 = fig.add_subplot(gs[3, 2])
    window = 20
    real_vol = pd.Series(real).rolling(window).std()
    ax10.plot(real_vol[:500], label='Real', color='black', alpha=0.7,␣
↪linewidth=1.5)

    for idx, (name, syn_data) in enumerate(synthetic_dict.items()):
        syn = np.asarray(syn_data).flatten()
        syn_vol = pd.Series(syn).rolling(window).std()
        ax10.plot(syn_vol[:500], label=name, color=colors[idx+1], alpha=0.6,␣
↪linewidth=1)

    ax10.set_title(f'Rolling Volatility (window={window})', fontsize=13,␣
↪fontweight='bold', pad=10)
    ax10.set_xlabel('Time', fontsize=11)
    ax10.set_ylabel('Volatility (Std Dev)', fontsize=11)
    ax10.legend(loc='best', fontsize=9)
    ax10.grid(True, alpha=0.3)
    ax10.tick_params(labelsize=9)
```

```
    plt.suptitle('Stylized Facts Comparison: GANs vs Real Financial Returns',
                 fontsize=18, fontweight='bold', y=0.995)

    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
        print(f"Figure saved to {save_path}")

    plt.tight_layout()
    return fig


def plot_metrics_heatmap(results_df, save_path=None):
    """
    Create heatmap of metrics across models.

    Args:
        results_df: DataFrame from evaluate_multiple_models
        save_path: Path to save figure
    """
    # Select key metrics
    metric_cols = [col for col in results_df.columns
                   if col not in ['model', 'avg_rank'] and not col.
 ↪endswith('_rank')]

    # Normalize metrics to [0, 1] for visualization
    df_norm = results_df[metric_cols].copy()
    for col in df_norm.columns:
        min_val = df_norm[col].min()
        max_val = df_norm[col].max()
        if max_val > min_val:
            df_norm[col] = (df_norm[col] - min_val) / (max_val - min_val)

    df_norm.index = results_df['model']

    # Create heatmap
    fig, ax = plt.subplots(figsize=(14, 6))
    sns.heatmap(df_norm.T, annot=True, fmt='.3f', cmap='RdYlGn_r',
                cbar_kws={'label': 'Normalized Score (lower is better)'},
                linewidths=0.5, ax=ax)
    ax.set_title('Model Performance Heatmap (Normalized Metrics)', fontsize=14,␣
 ↪fontweight='bold')
    ax.set_xlabel('Model', fontsize=12)
    ax.set_ylabel('Metric', fontsize=12)

    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
        print(f"Heatmap saved to {save_path}")
```

```
    plt.tight_layout()
    return fig


def plot_rank_comparison(results_df, save_path=None):
    """
    Bar plot comparing average ranks across models.

    Args:
        results_df: DataFrame from evaluate_multiple_models
        save_path: Path to save figure
    """
    fig, ax = plt.subplots(figsize=(10, 6))

    models = results_df['model']
    avg_ranks = results_df['avg_rank']

    colors = plt.cm.RdYlGn_r(avg_ranks / avg_ranks.max())
    bars = ax.barh(models, avg_ranks, color=colors, edgecolor='black',␣
 ↪linewidth=1.5)

    # Add value labels
    for bar in bars:
        width = bar.get_width()
        ax.text(width, bar.get_y() + bar.get_height()/2,
                f'{width:.2f}', ha='left', va='center', fontsize=10,␣
 ↪fontweight='bold')

    ax.set_xlabel('Average Rank (lower is better)', fontsize=12,␣
 ↪fontweight='bold')
    ax.set_title('Model Performance: Average Rank Across All Metrics',
                 fontsize=14, fontweight='bold')
    ax.invert_yaxis()
    ax.grid(True, alpha=0.3, axis='x')

    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
        print(f"Rank comparison saved to {save_path}")

    plt.tight_layout()
    return fig


# Convenience wrapper
def create_all_plots(real_data, synthetic_dict, results_df=None,
                     output_dir=None, dataset_name=''):
```

```
    """
    Create all comparison plots.

    Args:
        real_data: Real returns
        synthetic_dict: {model_name: synthetic_data}
        results_df: Results DataFrame (optional)
        output_dir: Directory to save plots
        dataset_name: Name of dataset for filenames
    """
    if output_dir:
        import os
        os.makedirs(output_dir, exist_ok=True)
        prefix = f"{output_dir}/{dataset_name}_" if dataset_name else␣
↪f"{output_dir}/"
    else:
        prefix = None

    # Main stylized facts plot
    save_path = f"{prefix}stylized_facts.png" if prefix else None
    fig1 = plot_stylized_facts_comparison(real_data, synthetic_dict,␣
↪save_path=save_path)
    plt.show()

    # Metrics heatmap
    if results_df is not None:
        save_path = f"{prefix}metrics_heatmap.png" if prefix else None
        fig2 = plot_metrics_heatmap(results_df, save_path=save_path)
        plt.show()

        # Rank comparison
        save_path = f"{prefix}rank_comparison.png" if prefix else None
        fig3 = plot_rank_comparison(results_df, save_path=save_path)
        plt.show()

    return fig1
```

0.0.7 Integrated Pipeline

[9]:
```
# ============================================================================
# SETUP DIRECTORIES
# ============================================================================
DATA_DIR = os.getcwd()
TRAIN_DIR = os.path.join(DATA_DIR, "train")
VAL_DIR = os.path.join(DATA_DIR, "valid")
TEST_DIR = os.path.join(DATA_DIR, "test")
```

```python
def list_datasets(path):
    return sorted([os.path.join(path, f) for f in os.listdir(path) if f.
 ↪endswith(".parquet")])

train_datasets = list_datasets(TRAIN_DIR)
val_datasets = list_datasets(VAL_DIR)
test_datasets = list_datasets(TEST_DIR)

print("Train:", train_datasets)
print("Val:  ", val_datasets)
print("Test: ", test_datasets)

# ==============================================================================
# DATA LOADING
# ==============================================================================
def load_dataset(path):
    df = pd.read_parquet(path)
    print(f"Loaded {path}: shape={df.shape}, NaN count={df.isna().sum().sum()}")
    return df.to_numpy().astype(np.float32)

# ==============================================================================
# PROPER TIME SERIES POOLING
# ==============================================================================
def load_and_pool_datasets(dataset_paths, seq_len=128):
    """
    Load datasets and create windows FIRST, then pool windows.
    This preserves temporal structure within each window while allowing
    the model to learn from multiple markets.

    Args:
        dataset_paths: List of paths to training datasets
        seq_len: Sequence length for windowing

    Returns:
        Pooled windows array of shape (total_windows, seq_len, n_features)
    """
    all_windows = []
    total_original_samples = 0

    print("\n" + "="*80)
    print(" CREATING WINDOWS FROM EACH DATASET")
    print("="*80)

    for path in dataset_paths:
        dataset_name = os.path.basename(path).replace('_train.parquet', '')
        data = load_dataset(path)
```

```
        if data.ndim == 1:
            data = data[:, None]

        T, n_features = data.shape
        total_original_samples += T

        # Create windows from THIS dataset (maintains temporal order within␣
↪windows)
        if T >= seq_len:
            windows = []
            for i in range(T - seq_len + 1):
                windows.append(data[i:i+seq_len])

            windows = np.array(windows)
            all_windows.append(windows)

            print(f"  {dataset_name:15s}: {T:5d} samples → {len(windows):5d}␣
↪windows")
        else:
            print(f"  {dataset_name:15s}: {T:5d} samples → SKIPPED (too short)")

    if len(all_windows) == 0:
        raise ValueError("No valid datasets for windowing!")

    # Concatenate all windows
    pooled_windows = np.concatenate(all_windows, axis=0)

    # Shuffle windows (temporal order preserved WITHIN each window)
    np.random.seed(42)  # For reproducibility
    np.random.shuffle(pooled_windows)

    print(f"\n{'='*80}")
    print(f"  Successfully pooled {len(dataset_paths)} datasets")
    print(f"  Original total samples: {total_original_samples}")
    print(f"  Total windows created:  {len(pooled_windows)}")
    print(f"  Window shape:           {pooled_windows.shape}")
    print(f"  Data range:             [{pooled_windows.min():.4f},␣
↪{pooled_windows.max():.4f}]")
    print(f"{'='*80}\n")

    return pooled_windows

# ============================================================================
# OPTION 2: TRAIN SEPARATE MODEL PER DATASET
# ============================================================================
def train_models_per_dataset(model_class, model_config, train_datasets):
    """
```

```
    Train separate model instances for each dataset.
    Returns: dict {dataset_name: trained_model}
    """
    trained_models = {}

    for ds_path in train_datasets:
        dataset_name = os.path.basename(ds_path).replace('_train.parquet', '')
        print(f"\n{'='*80}")
        print(f"Training {model_class.__name__} on {dataset_name}")
        print(f"{'='*80}")

        # Create fresh model instance
        model = model_class(config=model_config)

        # Load and train
        data = load_dataset(ds_path)
        model.train(data)

        # Store
        trained_models[dataset_name] = model
        print(f"  {dataset_name} model trained")

    return trained_models

# ============================================================================
# TRAINING PIPELINE
# ============================================================================
def train_all_gans_pooled(train_datasets):
    """
    Train ONE model per GAN type using WINDOWED pooled data from all datasets.
    Windows maintain temporal structure, avoiding fake transitions.
    """
    print("\n" + "="*80)
    print(" TRAINING STRATEGY: POOLED WINDOWS (ONE MODEL PER GAN)")
    print("="*80)

    # Pool windows from all training data (temporal structure preserved)
    pooled_windows = load_and_pool_datasets(train_datasets, seq_len=128)

    # Since data is already windowed, we need to pass it directly to models
    # Models will handle their own windowing, so we flatten back to 1D time␣
 ↪series
    # BUT: we keep windows separate for proper training

    # Actually, we should modify model training to accept pre-windowed data
    # For now, let's flatten strategically: use first window from each
    # No wait - better solution: pass all windows as if they're one long series
```

```
# BEST: Just pass the pooled windows directly - models handle normalization
# We need to reshape: (n_windows, seq_len, n_features) → model expects this

# Define model configurations
models_config = {
    'TimeGAN': {
        "hidden_dim": 24,
        "noise_dim": 24,
        "num_layers": 3,
        "lr": 1e-3,
        "epochs": 5,
        "batch_size": 128
    },
    'QuantGAN': {
        "lr_g": 1e-4,
        "lr_d": 1e-4,
        "epochs": 5,
        "noise_dim": 100,
        "n_critic": 3,
        "lambda_gp": 10.0,
        "batch_size": 8192
    },
    'FinGAN': {
        "lr_g": 1e-4,
        "lr_d": 1e-4,
        "epochs": 5,
        "noise_dim": 100,
        "n_critic": 3,
        "lambda_gp": 10.0,
        "batch_size": 64
    }
}

# For training, we flatten the pooled windows back into a pseudo-time-series
# Each model will re-window, but the point is: we trained on data from ALL␣
↪markets
training_data = pooled_windows.reshape(-1, pooled_windows.shape[-1])

print(f"Training data shape for models: {training_data.shape}")
print(f"(Models will create their own windows from this)\n")

# Train each GAN on pooled data
trained_models = {}

print("\n" + "="*80)
print(" TRAINING TIMEGAN")
```

```
    print("="*80)
    timegan = TimeGAN(config=models_config['TimeGAN'])
    timegan.train(training_data)
    trained_models['TimeGAN'] = timegan

    print("\n" + "="*80)
    print(" TRAINING QUANTGAN")
    print("="*80)
    quantgan = QuantGAN(config=models_config['QuantGAN'])
    quantgan.train(training_data)
    trained_models['QuantGAN'] = quantgan

    print("\n" + "="*80)
    print(" TRAINING FINGAN")
    print("="*80)
    fingan = FinGAN(config=models_config['FinGAN'])
    fingan.train(training_data)
    trained_models['FinGAN'] = fingan

    return trained_models

# ============================================================================
# COMPREHENSIVE EVALUATION
# ============================================================================
def comprehensive_evaluation(trained_models, test_datasets,␣
 ↪output_dir='results'):
    """
    Comprehensive evaluation with all stylized facts metrics and visualizations.

    Args:
        trained_models: dict {model_name: trained_model} OR list of models
        test_datasets: list of test dataset paths
        output_dir: directory to save results
    """
    # These classes should already be defined in your notebook from the␣
 ↪artifacts
    # FinancialMetrics and plotting functions

    Path(output_dir).mkdir(parents=True, exist_ok=True)

    # Convert list to dict if needed
    if isinstance(trained_models, list):
        trained_models = {model.name(): model for model in trained_models}

    all_results = {}

    # ========================================
```

```python
# EVALUATE ON EACH TEST DATASET
# ========================================
for test_path in test_datasets:
    dataset_name = os.path.basename(test_path).replace('_test.parquet', '')

    print(f"\n{'='*80}")
    print(f" EVALUATING ON: {dataset_name}")
    print(f"{'='*80}\n")

    # Load real test data
    real_data = load_dataset(test_path).flatten()

    # Generate synthetic data from each model
    synthetic_dict = {}
    for model_name, model in trained_models.items():
        print(f"Generating synthetic data from {model_name}...")
        try:
            synthetic = model.generate(len(real_data))
            synthetic_dict[model_name] = synthetic.flatten()
            print(f"    Generated {len(synthetic.flatten())} samples")
        except Exception as e:
            print(f"    Failed: {e}")

    if len(synthetic_dict) == 0:
        print(f"Skipping {dataset_name} - no synthetic data generated")
        continue

    # ========================================
    # COMPUTE COMPREHENSIVE METRICS
    # ========================================
    print(f"\n{'-'*80}")
    print(" COMPUTING STYLIZED FACTS METRICS")
    print(f"{'-'*80}\n")

    results_df = evaluate_multiple_models(
        real_data,
        synthetic_dict,
        max_lag=min(50, len(real_data)//4)
    )

    # Save metrics
    metrics_path = f"{output_dir}/{dataset_name}_metrics.csv"
    results_df.to_csv(metrics_path, index=False)
    print(f"\n  Metrics saved: {metrics_path}")

    # ========================================
    # DETAILED ANALYSIS PER MODEL
```

```
        # ========================================
        detailed_metrics = {}
        for model_name, synthetic in synthetic_dict.items():
            print(f"\n{'-'*80}")
            print(f" DETAILED ANALYSIS: {model_name} on {dataset_name}")
            print(f"{'-'*80}")

            evaluator = FinancialMetrics(real_data, synthetic)
            metrics = evaluator.compute_all_metrics(max_lag=min(50,␣
↪len(real_data)//4))
            evaluator.print_summary(metrics)

            detailed_metrics[model_name] = metrics

        # ========================================
        # CREATE VISUALIZATIONS
        # ========================================
        print(f"\n{'-'*80}")
        print(" CREATING PLOTS")
        print(f"{'-'*80}\n")

        create_all_plots(
            real_data,
            synthetic_dict,
            results_df,
            output_dir=output_dir,
            dataset_name=dataset_name
        )

        print(f"  Plots saved: {output_dir}/{dataset_name}_*.png")

        # Store results
        all_results[dataset_name] = {
            'summary_metrics': results_df,
            'detailed_metrics': detailed_metrics,
            'synthetic_data': synthetic_dict
        }

    # ========================================
    # CROSS-DATASET SUMMARY
    # ========================================
    print(f"\n{'='*80}")
    print(" CROSS-DATASET SUMMARY")
    print(f"{'='*80}\n")

    # Aggregate results
    summary_rows = []
```

```python
    for dataset_name, results in all_results.items():
        df = results['summary_metrics']
        for _, row in df.iterrows():
            summary_rows.append({
                'dataset': dataset_name,
                **row.to_dict()
            })

    summary_df = pd.DataFrame(summary_rows)

    # Compute average performance per model
    key_metrics = ['avg_rank', 'mse', 'mae', 'wasserstein',
                   'kurtosis_diff', 'acf_absolute_mae', 'tail_index_diff']

    model_performance = summary_df.groupby('model')[key_metrics].mean().round(4)
    model_performance = model_performance.sort_values('avg_rank')

    print("\n" + "="*80)
    print(" OVERALL MODEL PERFORMANCE (Averaged Across Datasets)")
    print("="*80)
    print(model_performance.to_string())
    print("="*80 + "\n")

    print(f"\n BEST MODEL: {model_performance.index[0]}")
    print(f"   Average Rank: {model_performance.iloc[0]['avg_rank']:.2f}")

    # Save summaries
    summary_path = f"{output_dir}/overall_summary.csv"
    summary_df.to_csv(summary_path, index=False)

    performance_path = f"{output_dir}/model_performance_summary.csv"
    model_performance.to_csv(performance_path)

    print(f"\n  Summary saved: {summary_path}")
    print(f"  Performance saved: {performance_path}")

    return all_results, summary_df, model_performance

# ============================================================================
# VALIDATION FUNCTION (OPTIONAL - for hyperparameter tuning)
# ============================================================================
def validate_models(trained_models, val_datasets):
    """
    Validate trained models on validation datasets.
    """
    if isinstance(trained_models, list):
        trained_models = {model.name(): model for model in trained_models}
```

```python
    results = []

    for val_path in val_datasets:
        dataset_name = os.path.basename(val_path).replace('_val.parquet', '')
        data = load_dataset(val_path)

        print(f"\n  Validating on {dataset_name}")

        for model_name, model in trained_models.items():
            try:
                score = model.validate(data)
                print(f"  {model_name}: {score:.4f}")

                results.append({
                    "dataset": dataset_name,
                    "model": model_name,
                    "score": score
                })
            except Exception as e:
                print(f"  {model_name}: Failed - {e}")

    return pd.DataFrame(results)

# ============================================================================
# MAIN EXECUTION PIPELINE
# ============================================================================
def run_complete_pipeline():
    """
    Complete pipeline: train models, validate, test, and evaluate.
    """
    print("\n" + "="*80)
    print(" SYNTHETIC FINANCIAL TIME SERIES GENERATION")
    print(" GANs Comparison: TimeGAN vs QuantGAN vs FinGAN")
    print("="*80)

    # ========================================
    # STEP 1: TRAIN MODELS (POOLED DATA)
    # ========================================
    print("\n" + "="*80)
    print(" STEP 1: TRAINING MODELS")
    print("="*80)

    trained_models = train_all_gans_pooled(train_datasets)

    print("\n  All models trained successfully!")
```

```
    # ========================================
    # STEP 2: VALIDATION (OPTIONAL)
    # ========================================
    print("\n" + "="*80)
    print(" STEP 2: VALIDATION")
    print("="*80)

    val_results = validate_models(trained_models, val_datasets)
    print("\nValidation Results:")
    print(val_results.to_string(index=False))

    # ========================================
    # STEP 3: COMPREHENSIVE EVALUATION
    # ========================================
    print("\n" + "="*80)
    print(" STEP 3: COMPREHENSIVE TESTING & EVALUATION")
    print("="*80)

    results, summary, performance = comprehensive_evaluation(
        trained_models,
        test_datasets,
        output_dir='thesis_results'
    )

    # ========================================
    # FINAL SUMMARY
    # ========================================
    print("\n" + "="*80)
    print(" PIPELINE COMPLETE!")
    print("="*80)
    print("\n All results saved to: thesis_results/")
    print("\n Files generated:")
    print("   - {dataset}_metrics.csv (comprehensive metrics)")
    print("   - {dataset}_stylized_facts.png (main plot)")
    print("   - {dataset}_metrics_heatmap.png")
    print("   - {dataset}_rank_comparison.png")
    print("   - overall_summary.csv")
    print("   - model_performance_summary.csv")

    print(f"\n Best Model: {performance.index[0]}")
    print(f"   Key Metrics:")
    best_model_stats = performance.iloc[0]
    for metric in ['avg_rank', 'mse', 'wasserstein', 'kurtosis_diff',␣
↪'acf_absolute_mae']:
        if metric in best_model_stats:
            print(f"     - {metric}: {best_model_stats[metric]:.4f}")
```

```
    return trained_models, results, summary, performance


# ==============================================================================
# JUPYTER NOTEBOOK USAGE
# ==============================================================================
"""
# In your Jupyter notebook, run:

# 1. Import all classes (TimeGAN, QuantGAN, FinGAN, BaseGAN)
# 2. Import metrics and plots modules
# 3. Run the complete pipeline:

trained_models, results, summary, performance = run_complete_pipeline()

# Or run step by step:

# Step 1: Train
trained_models = train_all_gans_pooled(train_datasets)

# Step 2: Validate (optional)
val_results = validate_models(trained_models, val_datasets)

# Step 3: Comprehensive evaluation
results, summary, performance = comprehensive_evaluation(
    trained_models,
    test_datasets,
    output_dir='thesis_results'
)

# Step 4: Access results
print(performance)  # Best model overall
summary  # All metrics per dataset
results  # Detailed results dictionary
"""
```

```
Train: ['/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/train/BOVESPA_train.parquet',
'/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/train/FTSE_train.parquet',
'/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/train/MSCI_train.parquet',
'/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/train/NIFTY50_train.parquet',
'/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/train/SHANGHAI_train.parquet']
Val:   ['/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/valid/BOVESPA_valid.parquet',
```

```
'/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/valid/FTSE_valid.parquet',
'/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/valid/MSCI_valid.parquet',
'/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/valid/NIFTY50_valid.parquet',
'/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/valid/SHANGHAI_valid.parquet']
Test:  ['/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/test/BOVESPA_test.parquet',
'/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/test/FTSE_test.parquet',
'/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/test/MSCI_test.parquet',
'/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/test/NIFTY50_test.parquet',
'/home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/test/SHANGHAI_test.parquet']
```

[9]:
```
"\n# In your Jupyter notebook, run:\n\n# 1. Import all classes (TimeGAN,
QuantGAN, FinGAN, BaseGAN)\n# 2. Import metrics and plots modules\n# 3. Run the
complete pipeline:\n\ntrained_models, results, summary, performance =
run_complete_pipeline()\n\n# Or run step by step:\n\n# Step 1:
Train\ntrained_models = train_all_gans_pooled(train_datasets)\n\n# Step 2:
Validate (optional)\nval_results = validate_models(trained_models,
val_datasets)\n\n# Step 3: Comprehensive evaluation\nresults, summary,
performance = comprehensive_evaluation(\n    trained_models, \n
test_datasets, \n    output_dir='thesis_results'\n)\n\n# Step 4: Access
results\nprint(performance)  # Best model overall\nsummary  # All metrics per
dataset\nresults  # Detailed results dictionary\n"
```

[ ]:
```
# This is the MAIN execution cell
trained_models, results, summary, performance = run_complete_pipeline()
```

```
================================================================================
 SYNTHETIC FINANCIAL TIME SERIES GENERATION
 GANs Comparison: TimeGAN vs QuantGAN vs FinGAN
================================================================================

================================================================================
 STEP 1: TRAINING MODELS
================================================================================

================================================================================
 TRAINING STRATEGY: POOLED WINDOWS (ONE MODEL PER GAN)
================================================================================
```

```
================================================================================
 CREATING WINDOWS FROM EACH DATASET
================================================================================
Loaded /home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/train/BOVESPA_train.parquet: shape=(996, 1), NaN
count=1
  BOVESPA        :   996 samples →   869 windows
Loaded /home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/train/FTSE_train.parquet: shape=(1000, 1), NaN count=1
  FTSE           :  1000 samples →   873 windows
Loaded /home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/train/MSCI_train.parquet: shape=(1004, 1), NaN count=1
  MSCI           :  1004 samples →   877 windows
Loaded /home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/train/NIFTY50_train.parquet: shape=(993, 1), NaN
count=1
  NIFTY50        :   993 samples →   866 windows
Loaded /home/sobottka/BSE/Master_Thesis/bse-thesis-synthetic-
data/data/processed_files/train/SHANGHAI_train.parquet: shape=(969, 1), NaN
count=1
  SHANGHAI       :   969 samples →   842 windows

================================================================================
  Successfully pooled 5 datasets
  Original total samples: 4962
  Total windows created:  4327
  Window shape:           (4327, 128, 1)
  Data range:             [nan, nan]
================================================================================

Training data shape for models: (553856, 1)
(Models will create their own windows from this)


================================================================================
 TRAINING TIMEGAN
================================================================================
[TimeGAN] WARNING: Found 576 NaN values, handling…
[TimeGAN] NaN handling complete
[TimeGAN] Creating windows: T=553856, seq_len=256, n_features=1
[TimeGAN] Data range: [-0.7768, 0.6546]
[TimeGAN] Mean: -0.022151, Std: 0.189897
[TimeGAN] Normalized range: [0.0000, 1.0000]
[TimeGAN] Created 553601 windows
[TimeGAN] Models built:
  Embedder: 9,744
  Recovery: 10,825
  Generator: 11,400
```

```
  Supervisor: 7,800
  Discriminator: 10,825
[TimeGAN] Phase 1: Training Autoencoder…
  Epoch 1/2 | Reconstruction Loss: 0.0200
[TimeGAN] Phase 2: Training Supervisor…
  Epoch 1/2 | Supervisor Loss: 0.0035
[TimeGAN] Phase 3: Joint Adversarial Training…
[TimeGAN] Epoch 1/5 | D=1.3837 | G=8.9708
[TimeGAN] Epoch 5/5 | D=1.3770 | G=2.2292

================================================================================
 TRAINING QUANTGAN
================================================================================
[QuantGAN] WARNING: Found 576 NaN values, handling…
[QuantGAN] NaN handling complete
[QuantGAN] Creating windows: T=553856, seq_len=256, n_features=1
[QuantGAN] Data range: [-0.7768, 0.6546]
[QuantGAN] Mean: -0.022151, Std: 0.189897
[QuantGAN] Normalized range: [-0.9900, 0.9900]
[QuantGAN] Created 553601 windows
[QuantGAN] Models built - G params: 483,329
[QuantGAN] Models built - D params: 115,489
[QuantGAN] Starting training with 67 batches per epoch
```